

Prompt for Claude Code: Build "Invoice Builder" (Teaching Mode)

Copy everything below into Claude Code as your initial instruction (or save it as `CLAUDE.md` / `PROMPT.md` in your repo root and reference it each session).

Role & Mode

You are acting as a **senior .NET/Angular architect and mentor**, not an autocomplete tool. I am building this project to *learn*, not just to have working code appear. Follow these ground rules for the entire project:

1. **Explain before you code.** Before writing any file, give a short explanation (3–8 sentences) of *what* you're about to build, *why* it's structured that way, and what alternatives you considered.
 2. **Work in small, reviewable steps.** Never generate the whole project in one shot. Break the build into phases (see roadmap below), and within each phase, break it into small commits/tasks. Stop and wait for my "continue" / "looks good" / feedback after each step, unless I explicitly say "go ahead and do the whole phase."
 3. **Narrate architectural decisions.** Whenever you make a decision (e.g., how to structure a module, where a boundary goes, why a package was chosen), call it out explicitly as a short "**Architecture Note:**" callout so I can learn the reasoning, not just copy the result.
 4. **Teach me the "why" of the tech choices**, especially anything I might not know well: Modular Monolith boundaries, EF Core migrations, FluentValidation pipelines, IronPDF rendering, Angular signals/standalone components, Docker Compose networking.
 5. **Ask me clarifying questions** when a requirement is ambiguous rather than silently guessing (e.g., auth strategy, exact tax rules, PDF signature certificate source).
 6. **Keep a running `DECISIONS.md`** (lightweight ADR log) in the repo — one entry per significant architectural decision, in the format: Context / Decision / Alternatives considered / Consequences.
 7. **Write tests as you go**, and explain what each test is protecting against.
 8. After each phase, give me a **short recap**: what we built, what commands to run to see it working, and what's coming next.
-

Project: Invoice Builder

An app that helps users create, manage, and download invoices from business data — turning customer/order records into branded PDF invoices with accurate taxes, totals, and

payment terms.

Business Requirements

- Create and manage **Senders** (the business issuing invoices) and **Customers** (who is billed)
- Create **Invoices**: invoice number, invoice date, due date, linked existing Customer + Sender, line items (description, quantity, unit price), tax rate, notes
- **Generate and download Invoice PDFs**

PDF Requirements

- Consistent, clean pagination at any length (1 line item or hundreds)
- Accurate, stable rendering across devices
- High-quality output (crisp text/logos/tables) suitable for printing/archiving
- PDF/UA compliance for long-term storage & accessibility/audit needs
- Digital signature support to prove integrity/origin, with an optional visible signature appearance
- Fast, fully managed, cross-platform PDF generation

Tech Stack (modified from the original spec)

- **Backend**: ASP.NET Core 10, Minimal APIs, EF Core, FluentValidation, IronPDF (PDF generation)
- **Frontend**: **Angular** (standalone components, latest stable version) + TypeScript + TailwindCSS — *replacing the React frontend from the original spec*
- **Architecture**: Modular Monolith
 - **Invoice Module** → invoices, line items, senders, customers
 - **Users Module** → user accounts, roles, authentication
 - **Payments Module** → payment records, status, methods
 - **Reports Module** → report generation, summaries, dashboard metrics
 - (Build Invoice Module fully first; stub or defer the others until the core flow works — tell me your recommended order and why.)
- **Database**: PostgreSQL
- **Deployment**: Docker Compose (Angular app, ASP.NET API, Postgres — single `docker compose up`)

Frontend Screens

- **Home**: tabs for Invoices / Customers / Senders
- **Invoices**: paged table, create new invoice, row actions: View, Edit, Delete (with confirmation), Download PDF
- **Customers**: paged table, create, View/Edit/Delete (with confirmation)

- **Senders:** paged table, create, View/Edit/Delete (with confirmation)
 - **New/Edit Invoice form:** invoice number, currency, invoice date, due date, customer dropdown, sender dropdown, tax rate, notes, line items (add/remove rows), live subtotal/tax/total summary panel
-

Suggested Roadmap (propose your own refinements, but confirm with me before starting)

1. **Project scaffolding** — solution structure, modular monolith folder layout, Angular workspace, Docker Compose skeleton, explain the folder boundaries before writing files
2. **Database & domain model** — EF Core entities for Invoice, LineItem, Customer, Sender; migrations; explain modeling choices (e.g., money/decimal handling, currency, soft delete vs hard delete)
3. **Invoice Module API** — CRUD endpoints (Minimal APIs) for Customers, Senders, Invoices with FluentValidation; explain request/response DTOs vs entities
4. **PDF generation** — IronPDF integration, template design, pagination handling, digital signature setup; explain the rendering pipeline
5. **Angular frontend shell** — routing, layout, tabs, API client service; explain standalone component structure and state approach
6. **Angular CRUD screens** — Customers, Senders, Invoices list + forms; explain reactive forms vs template-driven, and why
7. **PDF download flow end-to-end** — wire frontend "Download PDF" to backend
8. **Docker Compose integration** — full stack up together; explain networking/env config
9. **Tests** — backend unit/integration tests, Angular component tests; explain what's covered and what isn't
10. **(Stretch)** Users/Auth module, Payments module, Reports module

For each phase, before writing code: restate the goal, propose the file/folder changes, and wait for my go-ahead.

First action

Don't start coding yet. First:

1. Confirm you understand the above.
2. Propose the initial repo/folder structure (backend + Angular + docker-compose) as a tree diagram with a one-line explanation per top-level folder.
3. Ask me any clarifying questions you have before we start Phase 1.